

Department of Information Technology

Deep Learning Lab Project (Sem-VII)

Semester VII: Academic Year 2026-27

FoodLens

Smart Food Recognition Using Deep Learning

With Top-3 Predictions and Ingredient Information

Mini Project Synopsis

Project Guide: Dr. Sushopti Gawade

Hypothesis: Under a common data split and training budget, at least one ImageNet-pretrained convolutional neural network will achieve higher held-out food-classification accuracy than a custom CNN trained from scratch. This hypothesis will be tested experimentally, not assumed to be true.

Group Members:

Sr. No.	Full Name	Roll Number
1	Sarthak Kulkarni	23101B0019
2	Harshad Patekar	23101B0026
3	Tanay Shinde	23101B0034
4	Yash Patil	23101B0060

Abstract

FoodLens is a proposed image-based food recognition application that will identify the dominant dish in an uploaded photograph and display its three highest-scoring food categories. The project will use Food-101, a public benchmark containing 101,000 images from 101 food categories [4]. Five deep learning architectures—a custom CNN, ResNet50, MobileNetV2, DenseNet121, and EfficientNetB0—will be compared using a consistent experimental protocol. The selected model will be integrated into a web interface with prediction-score visualization and a curated dish-to-ingredient lookup. Ingredient information will describe common recipes rather than claim to detect ingredients in the photograph. The project will evaluate recognition quality, model size, and inference time, and demonstrate low-score warnings and failure cases. Its intended contribution is a reproducible food-recognition prototype, not a new neural architecture or a medical nutrition tool.

Keywords: Food recognition, deep learning, Food-101, convolutional neural network, transfer learning, image classification, ingredient information.

Introduction

Food recognition is challenging because the same dish can vary in appearance while different dishes can share textures and colours [1, 3]. This project will investigate these challenges through a controlled model comparison and an interactive demonstration. The application will support food-image organization and educational exploration while clearly separating learned category predictions from manually curated food information.

1 Literature Review

The following three journal papers from 2023–2024 provide the foundation for the proposed work. Their findings are literature evidence, not results obtained by this mini project.

No.	Journal	Year	Title of the Paper	Authors	Takeaways / Methods	Remarks / Link
1	Food and Humanity, Vol. 1, pp. 800–808	2023	A comparative study of deep learning methods for food classification with images	G. Suddul; J. F. L. Seguin	Compares a CNN trained from scratch with transfer learning; uses augmentation and EfficientNetV2.	Supports a baseline-versus-transfer study. Uses 11 food groups, not our 101-class task. Publisher link [1].
2	Human-Centric Intelligent Systems, Vol. 4, pp. 171–186	2024	Automatic Food Recognition Using Deep Convolutional Neural Networks with Self-attention Mechanism	R. Abiyev; J. Adepoju	Combines CNNs, self-attention, and averaged ensemble predictions; evaluates Food-101 and MA Food-121.	Motivates analysis of confusing classes. Attention and ensembles remain outside the core scope. Publisher link [2].
3	Computers in Biology and Medicine, Vol. 175, Art. 108528	2024	Fine-grained food image classification and recipe extraction using a customized deep neural network and NLP	A. Razia Sulthana; T. Tilford; S. Stoyanov	Uses MResNet-50 for classification and NLP with a domain ontology for ingredient and recipe processing.	Motivates a separate information layer. Our curated lookup does not reproduce its NLP pipeline. Publisher link [3].

Project Positioning

FoodLens will combine a five-model benchmark, top-3 score display, a low-score warning, and a transparent ingredient-information layer in a manageable undergraduate prototype. The claimed contribution is integration and evaluation, rather than algorithmic novelty or a guaranteed improvement over published research.

Problem Statement

Given one RGB photograph containing a dominant food item, predict its category from a fixed set of supported classes and present clearly qualified information about the prediction. The system must not present recipe-level ingredients as image-verified facts.

Objectives

1. Prepare a reproducible Food-101 training, validation, and test pipeline.
2. Implement and compare at least five deep learning architectures.
3. Display the top-3 predictions, model scores, and low-score warnings.
4. Retrieve curated common ingredients for supported predicted dishes.
5. Deliver a web demonstration, evaluation report, and documented limitations.

2 Data Set

Data set: Food-101, introduced by Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool at ECCV 2014 [4].

Official data set link: https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/

Data summary: 101 categories with 1,000 images per category. The official split supplies 750 training images and 250 manually reviewed test images per category. Training images intentionally include noise, including some incorrect labels. Images have a maximum side length of 512 pixels [4, 5].

Table of Data Set Parameters

Parameter / Field	Description
Image identifier / path	Relative path to the image, used to load and track each sample.
Image pixels	RGB image used as input; resized to $224 \times 224 \times 3$ for this project.
Food category	One of 101 dish labels; the target variable for classification.
Class index	Project-generated integer from 0 to 100, with a saved index-to-label mapping.
Official split	Training or test membership from the supplied metadata.
Validation membership	Project-generated subset drawn only from official training images.
Example class labels	Apple pie, pizza, sushi, ramen, hamburger, and ice cream.
Ingredient annotations	Not used as image-level targets; common ingredients will be stored in a separate curated lookup.

Proposed Data Partition

Within each category, 150 of the 750 official training images will be reserved for validation using a fixed seed. The official test set will remain untouched during development.

Partition	Per Class	Total	Purpose
Training	600	60,600	Learn network parameters
Validation	150	15,150	Select models and warning threshold
Test	250	25,250	Final held-out evaluation

Preparation and Feasibility

The pipeline will check file readability, convert images to RGB, save split manifests, and audit exact duplicates across partitions. Any necessary exclusions will be recorded with revised counts. Random augmentation will apply only to training images; validation and test preprocessing will be deterministic. Each pretrained backbone will receive its documented input preprocessing rather than a universal normalization rule [6].

A fixed 20-class subset may be used for initial debugging. The intended final comparison will cover all 101 classes. If compute limits require a smaller final scope, all five models will use the same declared subset and the report will not describe its results as a full Food-101 benchmark.

3 Methodology: System Flow Diagram

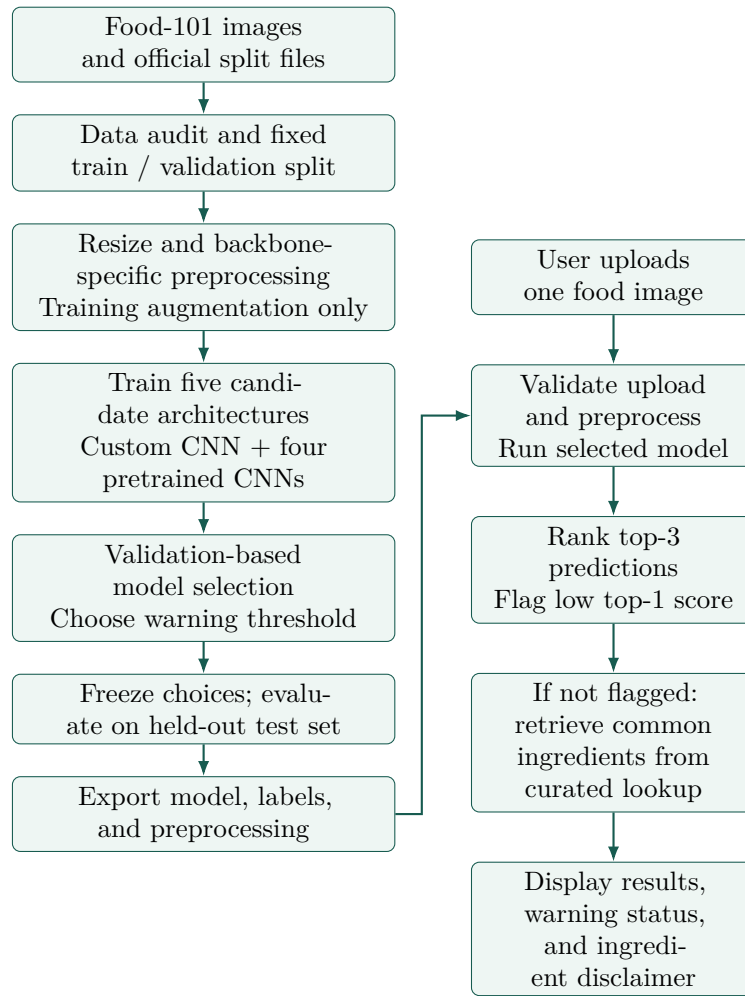


Figure 1: Training and inference workflow. Test data never guide model selection.

Processing Steps

1. **Prepare data:** establish the shared split and preprocessing pipeline.
2. **Train candidates:** fit the custom CNN; train new classification heads on frozen pretrained backbones, then fine-tune selected upper layers.
3. **Select and evaluate:** select by validation top-1 accuracy, using inference time as a tie-breaker; lock all choices before final testing.
4. **Deploy:** accept valid JPG/PNG uploads and run the saved inference pipeline.
5. **Present results:** display scores and retrieve common ingredients only when the prediction is not flagged as low-score and a lookup entry exists.

Warning policy: If the maximum model score is below a validation-selected threshold τ , the interface will say “Uncertain prediction—try a clearer image” and suppress ingredient suggestions. A high score is not proof that an image contains a supported food. The warning is not a validated food/non-food detector; network scores must not be treated as guaranteed probabilities of correctness [7].

4 Algorithm Details: Five Deep Learning Models

All five models will solve the same 101-class classification problem. The four pretrained architectures are available through Keras Applications [6]; their mechanisms are documented in the linked architecture references.

Algorithm	Core Method	Role in This Project
Custom CNN	Three convolution–ReLU–max-pooling blocks with 32, 64, and 128 filters; global average pooling; dense layer and dropout.	Train from random initialization as a baseline. Use 3×3 convolutions and a 128-unit dense layer.
ResNet50	Residual connections pass features through shortcut paths.	Evaluate residual feature extraction with an ImageNet-pretrained backbone.
MobileNetV2	Inverted residual blocks and depthwise convolutions support efficient feature extraction.	Evaluate the accuracy–latency trade-off for a lightweight demonstration.
DenseNet121	Dense connectivity reuses features from earlier layers.	Compare densely connected feature learning with residual learning.
EfficientNetB0	An architecture from a family designed around balanced network scaling.	Evaluate an alternative compact backbone for the application.

Model Design

For each pretrained model, the original classifier will be replaced by the same project-specific head. The models are alternatives; they will not be chained together or ensembled in the core implementation.

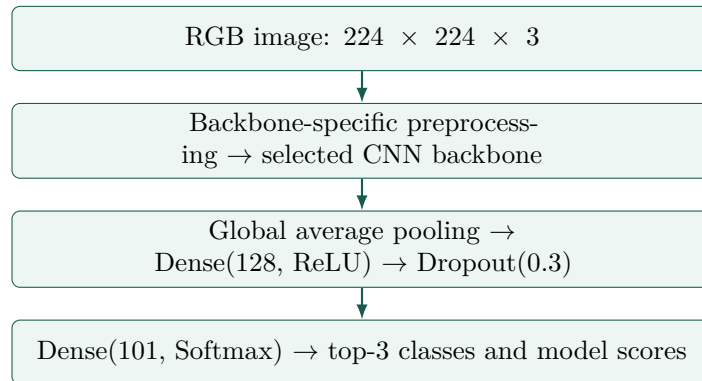


Figure 2: Proposed architecture for each transfer-learning experiment.

Learning Objective and Initial Settings

For an image x , class logits z_c , and true label y , the class score and training loss are

$$p_c(x) = \frac{\exp(z_c)}{\sum_{j=1}^{101} \exp(z_j)}, \quad \mathcal{L}(x, y) = -\log p_y(x).$$

Initial settings: Adam, mini-batch size 32, and learning rate 10^{-3} . Transfer models will receive up to 5 frozen-backbone epochs and 15 fine-tuning epochs at 10^{-5} ; the custom CNN will receive up to 20 epochs. Early stopping will monitor validation loss with patience 3 and restore the best checkpoint. Pretrained batch-normalization statistics will remain frozen. These settings are provisional; changes and actual training times will be reported.

5 Other Details: Five Project Modules

No.	Module	Responsibilities and Deliverables
1	Data preparation	Batches 1–10: Acquire Food-101, inspect samples, save splits, implement augmentation and model-specific preprocessing. Deliver reproducible loaders and a data summary.
2	Model training and comparison	Batches 11–20: Train all five architectures, save checkpoints and learning curves, and compare validation metrics. Deliver a selected model and experiment log.
3	Ingredient information	Batches 21–30: Create a curated JSON mapping from supported dishes to common ingredients and source references. Deliver a lookup service with recipe-variation warnings.
4	Prediction and score analysis	Batches 31–40: Return top-3 labels and scores, implement the warning policy, measure test performance, and inspect confusing classes. Deliver prediction utilities and plots.
5	Web application and integration	Batches 41–50: Build an upload interface, display predictions and ingredient information, validate inputs, and integrate the saved model. Deliver the demonstration and usage guide.

6 Evaluation Plan

- **Top-1 accuracy:** fraction of test images whose highest-scoring label is correct; the primary comparison metric.
- **Top-3 accuracy:** fraction whose true label appears among the three displayed predictions.
- **Macro precision, recall, and F1:** average class-level metrics to expose uneven performance; include a confusion matrix and per-class results.
- **Efficiency:** saved model size, trainable parameter count, training duration, and median single-image inference time after warm-up on the same documented device.
- **Warning behaviour:** report the proportion of predictions retained and their accuracy at the chosen threshold. Full-test accuracy will still include every test image.
- **Application checks:** verify valid uploads, invalid files, missing ingredient entries, low-score predictions, and consistent results after reloading the model.

The final report will compare all models on identical test images only after validation-based choices are fixed. A secondary set of user-provided food and non-food images will illustrate domain-shift failures, but will not be mixed into the Food-101 benchmark. Where resources permit, repeat the baseline and selected model with three random seeds and report mean and standard deviation.

Scope and Limitations

The system performs single-label classification, not multi-dish detection or segmentation. It will not estimate calories, portion size, freshness, or allergens. Ingredient suggestions are typical recipe information, not evidence of actual contents or suitability for a dietary restriction. Unsupported dishes and non-food objects can still receive high scores. Uploaded images will be processed transiently by default and will not be added to training data without consent.

7 Fifty-Batch Work Distribution

The project consists of **five modules with ten work batches per module**, giving

$$5 \text{ modules} \times 10 \text{ batches per module} = 50 \text{ work batches.}$$

A batch is a planned work package, not a training mini-batch or a separate student team. The four group members will share these packages; a batch is complete when its deliverable is reviewed and recorded. Global batch numbers run from 1 to 50, with local batch numbers 1 to 10 inside each module.

Execution order: Complete Module 1 before model training. Module 3 can begin after the class mapping is ready. Module 4 depends on the trained models and ingredient lookup; Module 5 completes integration and submission. Interface prototypes may begin earlier, but final acceptance follows the module handoffs.

7.1 Module 1: Data preparation

Work allocation: Batches 1–10 (10 batches).

Objective: Build a reproducible, audited dataset pipeline shared by all five models.

Inputs: Food-101 images, official split metadata, and the agreed project scope.

Global	Local	Assigned Work	Measurable Deliverable
1	1	Confirm scope and literature	Record the recognition task, supported classes, objectives, and takeaways from the three literature-review papers.
2	2	Acquire and inventory data	Prepare the image directory and official metadata; record dataset source and download checks.
3	3	Explore class distribution	Produce class counts and a labelled sample gallery; document visually similar food categories.
4	4	Audit image quality	Check unreadable files, image dimensions, and exact duplicates; save an audit and exclusion log.
5	5	Save label mapping	Create and verify a stable class-index mapping for training, ingredient lookup, and inference.
6	6	Create fixed partitions	Save stratified training and validation manifests from official training data; preserve the official test split.
7	7	Implement preprocessing	Build RGB conversion, resizing, and backbone-specific preprocessing; check expected tensor shapes.
8	8	Implement augmentation	Add training-only random transforms and inspect before/after examples for label preservation.
9	9	Build dataset loaders	Create batched loaders with training shuffle, deterministic evaluation, and bounded caching or prefetching.
10	10	Validate and hand off data	Verify counts, split separation, and reproducibility; release loaders, manifests, and a data preparation guide.

Module completion criterion: The saved splits and label mapping are reproducible, data issues are logged, and every candidate model can use the same dataset interface.

7.2 Module 2: Model training and comparison

Work allocation: Batches 11–20 (10 batches).

Objective: Train five candidate architectures and select the deployment model using validation data only.

Inputs: The audited loaders, split manifests, and label mapping from Module 1.

Global	Local	Assigned Work	Measurable Deliverable
11	1	Define comparison protocol	Record shared splits, epoch budgets, random seeds, validation metrics, and experiment naming rules.
12	2	Build training utilities	Implement checkpointing, early stopping, metric logging, and learning-curve export.
13	3	Check a small training run	Run short smoke tests on a fixed subset to verify loss, gradients, output dimensions, and label alignment.
14	4	Train the custom CNN	Train the from-scratch baseline and save its checkpoint, learning curves, and validation metrics.
15	5	Train ResNet50	Train the new classifier head, fine-tune upper layers, and save the validation results and checkpoint.
16	6	Train MobileNetV2	Repeat the agreed transfer-learning procedure and save the model, training log, and validation results.
17	7	Train DenseNet121	Repeat the agreed transfer-learning procedure and save the model, training log, and validation results.
18	8	Train EfficientNetB0	Repeat the agreed transfer-learning procedure and save the model, training log, and validation results.
19	9	Compare validation results	Create the five-model comparison, including validation accuracy, macro F1, model size, and training time.
20	10	Select and export the model	Select by the declared validation rule; export checkpoints and preprocessing metadata for Module 4. Keep test data unused.

Module completion criterion: Five completed model experiments share one documented protocol; the selected model and all comparison checkpoints are available without test-based selection.

7.3 Module 3: Ingredient information

Work allocation: Batches 21–30 (10 batches).

Objective: Create a source-recorded dish-to-ingredient knowledge layer without claiming image-level ingredient detection.

Inputs: The supported class names and label mapping from Module 1; this module does not require a trained model.

Global	Local	Assigned Work	Measurable Deliverable
21	1	Define lookup boundaries	Specify that entries describe common recipe ingredients and exclude image-verified or dietary-safety claims.
22	2	Design the data schema	Define JSON fields for class key, display name, common ingredients, recipe-variation note, and source references.
23	3	Create the class inventory	Create one lookup record per supported food class and track research and review status.
24	4	Collect recipe sources	Record traceable recipe references for the supported classes, including access dates.
25	5	Populate ingredient entries	Extract common ingredients into the draft records while preserving the source for each dish.
26	6	Normalize ingredient names	Standardize spelling and synonyms; remove duplicate ingredient names within each record.
27	7	Record recipe variation	Add notes for variable preparations and avoid presenting optional ingredients as certain contents.
28	8	Implement lookup functions	Map predicted class keys to validated records; provide a clear message when an entry is unavailable.
29	9	Test knowledge-layer behaviour	Check schema validity, class-key alignment, missing entries, and the required ingredient disclaimer.
30	10	Review and release the lookup	Complete a source and coverage review; release the JSON, lookup tests, and integration instructions.

Module completion criterion: Every supported class has a reviewed entry or an explicit unavailable status; every populated entry has source references and a recipe-variation disclaimer.

7.4 Module 4: Prediction and score analysis

Work allocation: Batches 31–40 (10 batches).

Objective: Provide reliable inference utilities, freeze the warning policy, and complete held-out evaluation.

Inputs: The checkpoints and metadata from Module 2 and the lookup interface from Module 3.

Global	Local	Assigned Work	Measurable Deliverable
31	1	Load inference artifacts	Verify model loading, class-index mapping, input shape, and backbone-specific preprocessing.
32	2	Implement single-image inference	Create a reusable prediction function returning the full class-score vector for one valid image.
33	3	Implement top-3 ranking	Return three ordered class labels and their model scores; test sorting and label alignment.
34	4	Create score visualizations	Produce labelled top-3 charts with consistent ordering and clear model-score terminology.
35	5	Select the warning threshold	Use validation data to choose the threshold and record retained coverage and accuracy; do not inspect test results.
36	6	Freeze the evaluation setup	Lock checkpoints, preprocessing, class mapping, threshold, and metric code before final testing.
37	7	Run held-out evaluation	Evaluate all five locked models on the same test images; report top-1, top-3, macro metrics, and the selected model's warning behaviour.
38	8	Analyse errors and efficiency	Produce confusion matrices, representative failures, and same-device inference timing after warm-up.
39	9	Test lookup and warning rules	Verify that low-score predictions suppress ingredients and that missing entries return a clear fallback message.
40	10	Release prediction services	Deliver tested inference utilities, score plots, a frozen evaluation report, and interface specifications for Module 5.

Module completion criterion: Prediction utilities reproduce saved-model results, warning and lookup rules pass tests, and final metrics are recorded without tuning on the test set.

7.5 Module 5: Web application and integration

Work allocation: Batches 41–50 (10 batches).

Objective: Integrate all components into a usable demonstration and complete final project delivery.

Inputs: The saved model, prediction utilities, ingredient lookup, and evaluation report from Modules 1–4.

Global	Local	Assigned Work	Measurable Deliverable
41	1	Specify the interface	Prepare an upload-to-result screen layout, expected user interactions, and acceptance checklist.
42	2	Build the application shell	Create the Streamlit page structure, configuration, and reusable model-loading path.
43	3	Implement upload validation	Accept supported JPG/PNG inputs, enforce a documented size limit, and show clear invalid-file messages.
44	4	Integrate prediction calls	Connect upload decoding and inference utilities with progress feedback and error handling.
45	5	Display recognition results	Show the uploaded image, predicted category, top-3 scores, and the low-score warning when applicable.
46	6	Integrate ingredient information	Display common ingredients, source references, variation notes, and missing-entry messages; obey suppression rules.
47	7	Check privacy and robustness	Test transient image handling, session isolation, malformed uploads, repeated requests, and model-load failures.
48	8	Run end-to-end acceptance tests	Verify representative food inputs, confusing classes, non-food failures, reload consistency, and the declared limitations.
49	9	Prepare the release package	Package source code, model-loading instructions, exact dependencies, reproducibility notes, and the final project report.
50	10	Deliver and demonstrate	Complete the presentation, demonstration script, and final checklist; archive the release and document any unresolved limitations.

Module completion criterion: The application passes its acceptance checklist; code, model artifacts or retrieval instructions, documentation, and presentation are ready for submission.

8 Resources and Project Completion

Completion rule: All 50 batch deliverables must be recorded as complete, all five module completion criteria must be met, and the integrated application must pass the final acceptance checklist. This synopsis describes planned work; it does not claim that these batches have already been executed.

Proposed Software and Hardware

Software plan: Python, TensorFlow/Keras for training, NumPy and pandas for analysis, Pillow for image handling, scikit-learn for metrics, Matplotlib/Seaborn for plots, and Streamlit for the interface. The implementation will record exact package versions and random seeds. These are project requirements, not a statement that models have already been trained.

Hardware planning estimate: a laptop with 8–16 GB RAM for development and inference, approximately 15 GB of free working storage, and access to a GPU with about 8 GB VRAM for training. Actual requirements depend on caching, checkpoints, and batch size; smaller batches will be used if needed. No paid service is required by the design.

Expected Outcomes

Deliverables will include a reproducible data pipeline, five-model comparison, saved validation-selected model, top-3 prediction interface, ingredient lookup, and failure analysis. The hypothesis will be supported only if measured results favour at least one pretrained model over the custom baseline. No accuracy values are claimed before experiments are completed.

Future Scope

Extensions include adding locally relevant dishes with newly labelled images, supervised multi-label ingredient prediction, food/non-food rejection trained on suitable examples, and mobile deployment. Each extension requires separate data preparation and evaluation.

References

- [1] G. Suddul and J. F. L. Seguin, “A comparative study of deep learning methods for food classification with images,” *Food and Humanity*, vol. 1, pp. 800–808, 2023. <https://doi.org/10.1016/j.foohum.2023.07.018>.
- [2] R. Abiyev and J. Adepoju, “Automatic Food Recognition Using Deep Convolutional Neural Networks with Self-attention Mechanism,” *Human-Centric Intelligent Systems*, vol. 4, pp. 171–186, 2024. <https://doi.org/10.1007/s44230-023-00057-9>.
- [3] A. Razia Sulthana, T. Tilford, and S. Stoyanov, “Fine-grained food image classification and recipe extraction using a customized deep neural network and NLP,” *Computers in Biology and Medicine*, vol. 175, article 108528, 2024. <https://doi.org/10.1016/j.combiomed.2024.108528>.
- [4] L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101—Mining Discriminative Components with Random Forests,” *European Conference on Computer Vision (ECCV)*, pp. 446–461, 2014. https://doi.org/10.1007/978-3-319-10599-4_29. Official data set: https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/.
- [5] PyTorch contributors, “Food101 dataset implementation,” *Torchvision documentation*. https://docs.pytorch.org/vision/2.0/_modules/torchvision/datasets/food101.html. Accessed September 5, 2026. Used to verify dataset structure, not as a requirement to use PyTorch.
- [6] Keras contributors, “Keras Applications,” official pretrained-model documentation and architecture references. <https://keras.io/api/applications/>. Model pages: ResNet, MobileNet, DenseNet, and EfficientNet. Accessed September 5, 2026.
- [7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *Proceedings of the 34th International Conference on Machine Learning*, PMLR, vol. 70, pp. 1321–1330, 2017. <https://proceedings.mlr.press/v70/guo17a.html>.